

Progetto di Simulazione di Sistemi

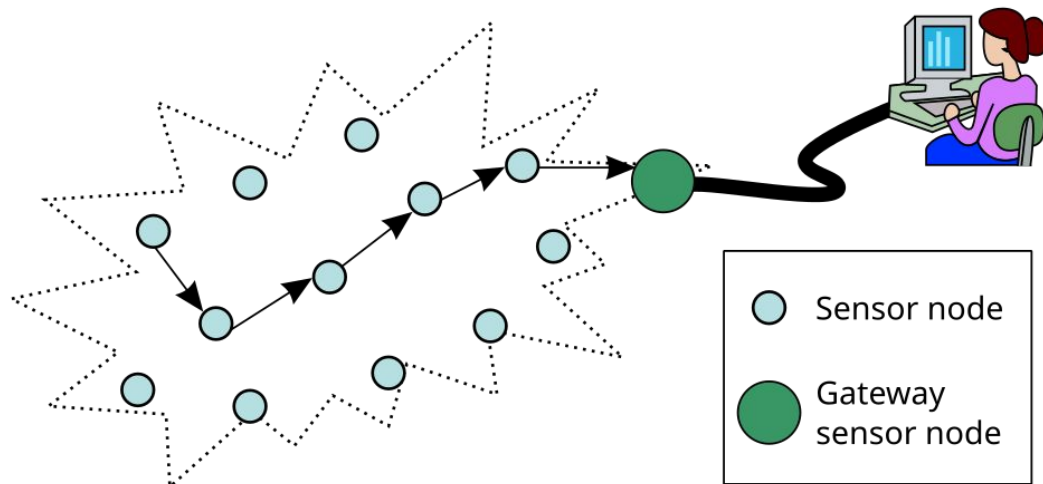
Beatrice Ravaglia
0001217838

LM Informatica 2025-2026

Introduzione

In molti sistemi reali, le entità non vengono processate una alla volta, bensì **in gruppi**.

- **Traffico stradale:** le automobili attraversano gli incroci in “ondate” di flusso.
- **Industria manifatturiera:** una macchina può lavorare solo quando ha accumulato un numero sufficiente di pezzi e può fermarsi in assenza di lavoro.
- **Wireless Sensor Networks (WSN):** i nodi trasmettono dati accumulati in blocchi per ottimizzare risorse e energia.





Descrizione del modello (1)

Il sistema simulato è costituito da:

- Due classi di utenti ($i = \{1, 2\}$).
- Una coda e un singolo server

Arrival process:

Il processo di arrivo degli utenti di classe i è di tipo batch.

Tempi di interarrivo:

$$A(t) \sim \text{Exp}(m) \quad m = \{1.0, 1.4, 2.0, 2.5\}$$

Dimensioni dei batch:

Ogni batch di classe i ha dimensione:

$$X_i \sim B_i(X_i)$$

$$B_1(X_1) \sim U(\{1, 2, 3, 4\}) \quad B_2(X_2) \sim U(\{1, 2, 3\})$$

Descrizione del modello (2)

Service Rule:

- Finché la coda contiene almeno a_1 utenti di classe 1:

$$r_1 = \min(b_1, \text{utenti di classe 1 in coda})$$

- Altrimenti, se ci sono almeno a_2 utenti di classe 2:

$$r_2 = \min(b_2, \text{utenti di classe 2 in coda})$$

- Altrimenti: **vacation**

Il **tempo di servizio** di ciascun batch dipende dalla dimensione del batch. Per un batch di dimensione r_i :

$$t_i = s_i + q_i, \text{ dove: } s_i = \frac{r_i}{b_i} sp_i \text{ e } q_i \sim U(l_i, u_i)$$

$a_1 = 7, b_1 = 16, sp_1 = 0.1, l_1 = 1.0, u_1 = 3.92$
--

$a_2 = 5, b_2 = 14, sp_2 = 0.1, l_2 = 1.0, u_2 = 2.54$
--

Descrizione del modello (3)

Il **tempo di vacation** dipende dal numero k di utenti di classe 1 presenti in coda:

$$t = t' + t'', \text{ dove: } t' = \frac{k+1}{b_1} sv \quad \text{e} \quad t'' \sim U(z, w)$$

Quando il server termina la fase di vacation:

- Se nella coda sono presenti almeno N utenti di classe 1 $\rightarrow$ fase di **Set Up**
- Altrimenti rimane in attesa finché soglia N raggiunta, poi Set Up

$$N = 20, sv = 0.2, z = 2.0, w = 3.8$$

Tempo di set up:

$$t \sim R(t) = \text{Exp}(h)$$

con $h = 1.0$



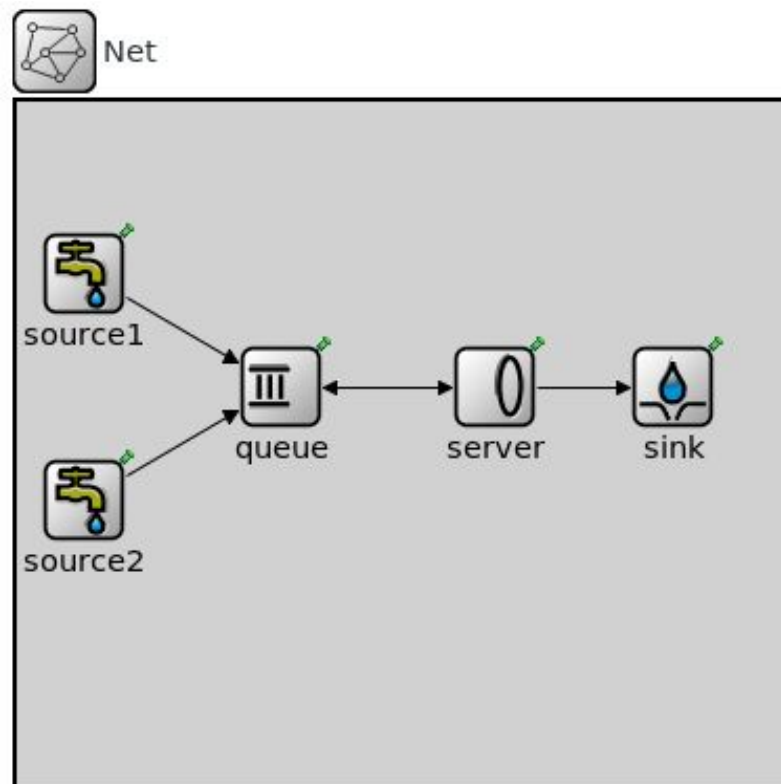
Implementazione



Il modello di simulazione è stato implementato tramite la piattaforma OMNeT++.

La struttura topologica della rete è stata definita in un file .ned, specificando i moduli che la compongono e le connessioni tra di essi.

I parametri del modello sono stati invece definiti nel file .ini di configurazione.



Componente Source e Job.msg

I batch di utenti appartenenti alle due differenti classi (1, 2) vengono generati tramite due moduli Source1 e Source2.



Metodi e messaggi principali:

- **handleMessage (cMessage * msg)**
 - **generateEvent**: self-message schedulato rispettando il tempo di inter-arrivo. Al verificarsi di tale evento, vengono creati tanti Job quanti specificati dalla dimensione del batch, assegnando a ciascuno la stessa classe utente e lo stesso timestamp di arrivo (simTime()). I job vengono quindi inviati verso la coda.
 - **SourceBatchComplete**: messaggio inviato alla coda per notificare il completamento dell'invio di un batch.

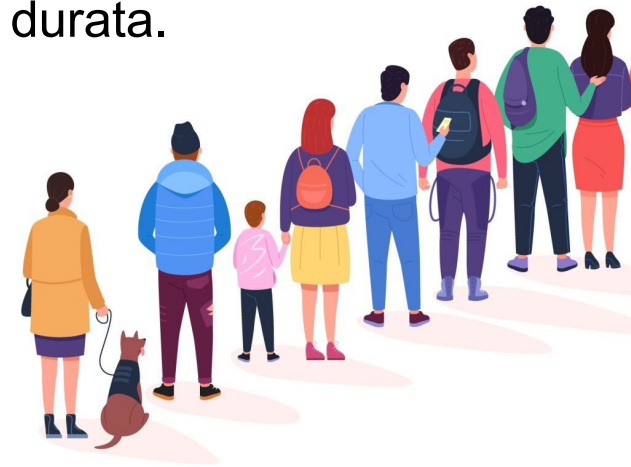
Per modellare le entità in ingresso al sistema è stato definito il messaggio **Job.msg**, che estende il tipo packet includendo due campi: userClass e arrivalTime.

Componente Queue

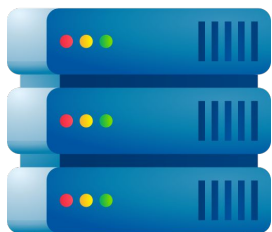
Mantiene due code distinte, `queueClass1` e `queueClass2`.

Metodi e messaggi principali:

- **handleMessage (cMessage * msg)**
Riceve e gestisce i messaggi provenienti da Source e Server.
- **serveBatch()**
Costruisce i batch da servire in base alla Service Rule, e invia i relativi Job al Server. Inoltre, calcola il tempo di servizio.
 - **BatchComplete**: messaggio inviato al server per notificare il completamento dell'invio di un batch.
- **startVacation()**
 - **StartVacation**: messaggio inviato al server per notificare l'inizio del periodo di vacation, insieme alla relativa durata.
- **startSetUp()**
 - **StartSetup**: messaggio inviato al server per notificare l'inizio del periodo di setup, insieme alla relativa durata.



Componente Server



Il componente Server è responsabile dell'esecuzione temporale delle attività di servizio, vacation e setup, mentre la logica decisionale (e il calcolo dei tempi) rimane delegata alla Queue.

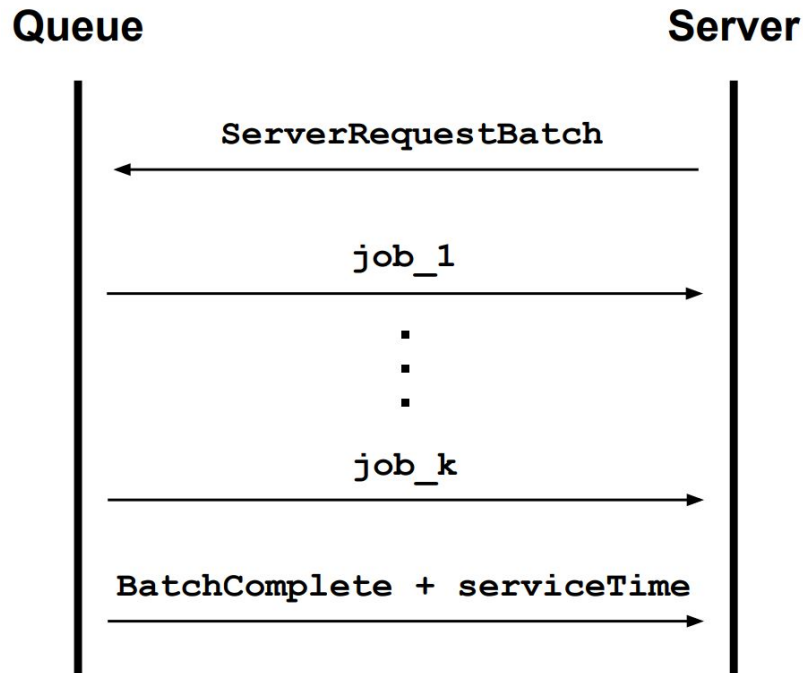
Metodi e messaggi principali:

- **handleMessage (cMessage * msg)**
 - **EndService**: self-message schedulato rispettando il tempo di servizio; dopo la sua ricezione, viene inviato un messaggio **JobDeparture** alla coda (per fini statistici), e tutti i job del batch servito vengono inviati al sink.
 - **ServerRequestBatch**: messaggio inviato alla coda per richiedere un nuovo batch da servire.
 - **VacationEnd**: self-message schedulato rispettando il tempo di vacation; dopo la sua ricezione, viene inviato un messaggio **ServerVacationEnded** alla coda.
 - **SetupEnd**: self-message schedulato rispettando il tempo di vacation; dopo la sua ricezione, viene richiesto un nuovo batch alla coda.



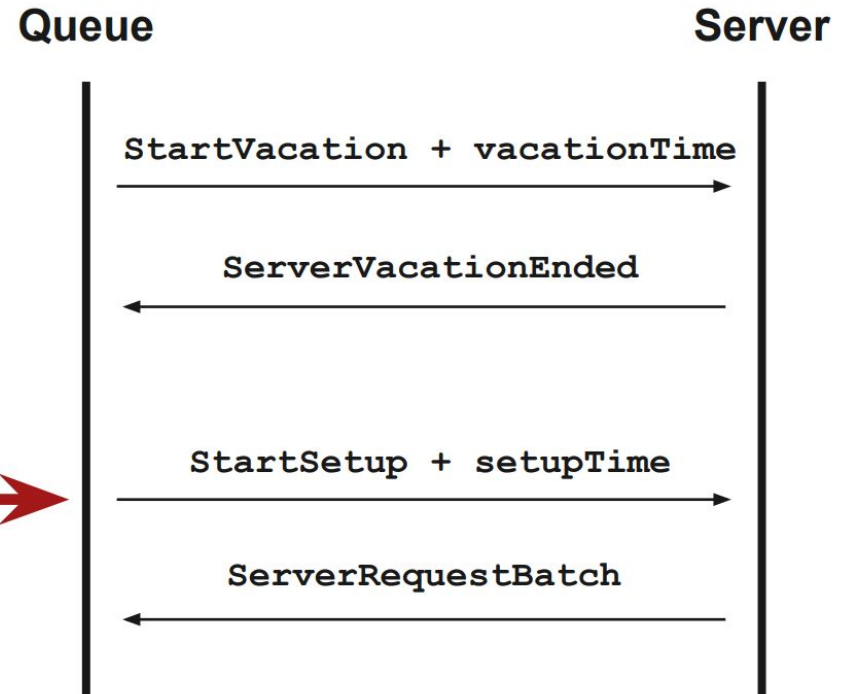
Interazione tra Queue e Server

Richiesta di un batch da servire



Fasi di vacation e set up

Quando la soglia N è raggiunta



Componente Sink

Il Sink rappresenta il punto di uscita del sistema.

Metodi e messaggi principali:

- **handleMessage (cMessage * msg)**
Elimina i job ricevuti dal Server, simulando la loro uscita definitiva dal sistema.





Raccolta di dati dalla simulazione

Sono stati raccolti i dati necessari per calcolare le stime puntuali e gli intervalli di confidenza delle seguenti misure di prestazione:

- Il **numero medio di utenti** di classe 1 e 2 presenti nel sistema → calcolato nella coda, tramite due vettori NumUsersSystemClass1 e NumUsersSystemClass2, aggiornati ogni volta che:
 - Arriva un nuovo batch di utenti dalla Source.
 - Viene ricevuto il messaggio “JobDeparture” dal Server, che indica l’uscita di un batch servito.
- Il **tempo medio di permanenza** nel sistema di un utente di classe 1 e 2 → registrato dal Sink nei vettori TimeInSystemClass1 e TimeInSystemClass2.
- Il **fattore di utilizzo del sistema** → alla fine di ogni periodo “busy”, la relativa durata viene registrata dal Server in un vettore Utilization.
- La **durata media dei periodi di vacation** → alla fine di ogni vacation, la relativa durata viene registrata dal Server in un vettore VacationDurations.

Analisi del transiente iniziale (1)

Durante la fase transiente, le misure di prestazione non rispecchiano il comportamento a regime del sistema. È quindi necessario individuare un opportuno periodo di warm up.

Metodo di Welch:

1. **Interpolazione** dei timestamp su una griglia temporale comune.
2. Per ogni metrica, per ogni valore X_r , si calcola la **media tra repliche** su ogni istante t :

$$\hat{\mu}(t) = \frac{1}{20} \sum_{r=1}^{20} X_r(t),$$

3. **Smoothing** per attenuare le fluttuazioni a breve termine:

$$\tilde{\mu}(t; K) = \begin{cases} \frac{1}{2K+1} \sum_{i=t-K}^{t+K} \hat{\mu}(i), & \text{per } t \geq K \\ \frac{1}{2t+1} \sum_{i=0}^{2t} \hat{\mu}(i), & \text{per } t < K \end{cases}$$

Analisi del transiente iniziale (2)

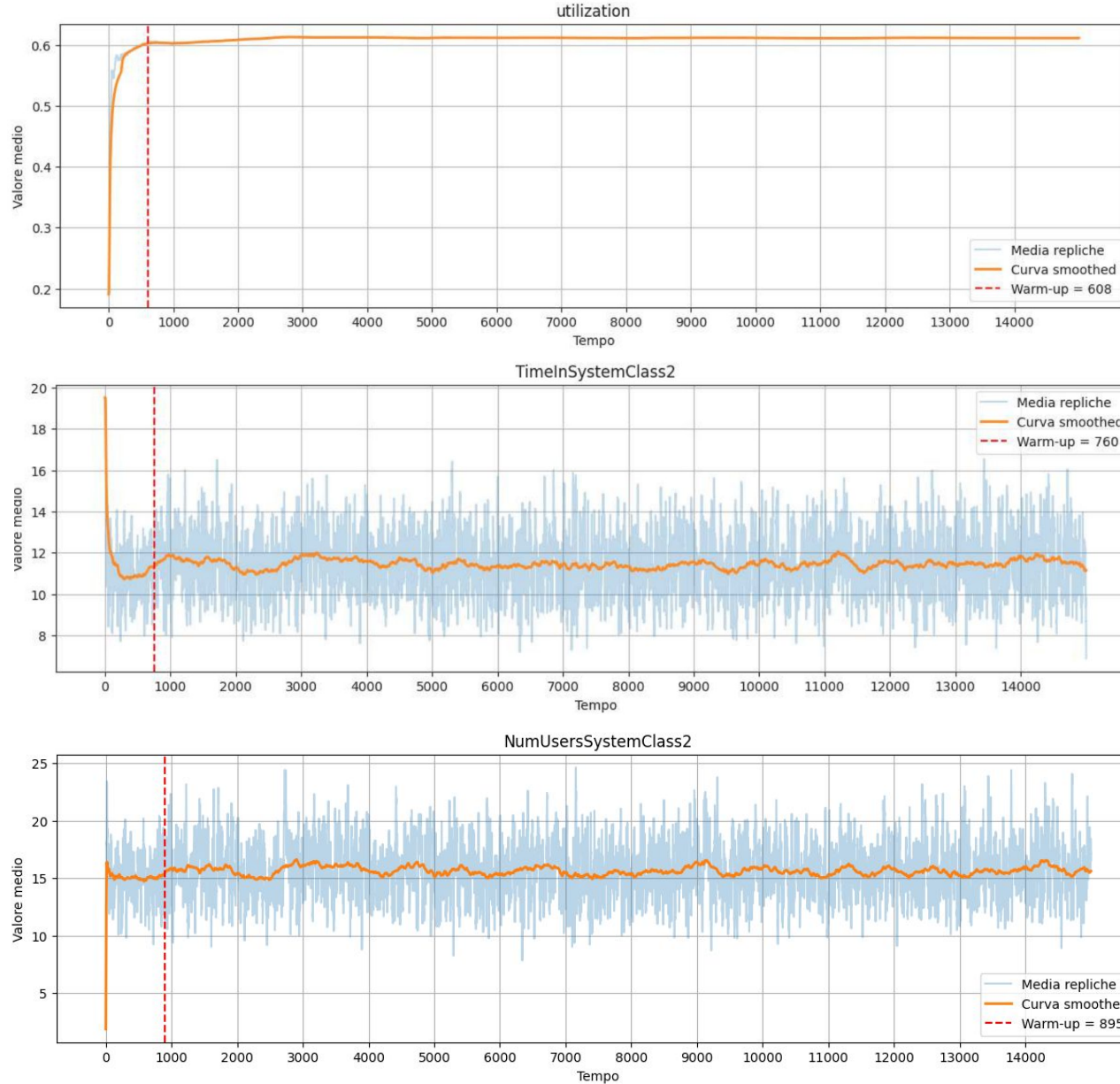
Tradizionalmente, il metodo di Welch è un approccio grafico; in questo lavoro, si è tentato un **approccio automatizzato**:

- La media stazionaria è stata stimata considerando l'ultimo 20% della curva smussata.
- È stata definita una banda di tolleranza pari all'1.5% della media stazionaria stimata.
- Il warm-up è stato individuato come il primo istante in cui la curva rimane stabilmente entro tale banda per una finestra temporale prefissata.

Poiché metriche diverse possono raggiungere il regime stazionario in istanti differenti, è stato assunto come tempo di fine transiente globale **il massimo tra i warm-up stimati** per ciascuna metrica, a cui è stato aggiunto un “margine di sicurezza” del 20%:

$$t_w^{\text{final}} = 1.2 \max_i t_w^{(i)}$$

Analisi del transiente iniziale (3)



Calcolo delle stime puntuali

Per ciascuna metrica, dopo la rimozione dei dati relativi al periodo di warm-up, è stata calcolata una stima Y_r per ciascuna run r .

- Fattore di utilizzo del sistema:

$$Y_r = \frac{\text{busy_time}_r}{T - t_w^{\text{final}}}$$

- Numero medio di utenti nel sistema:

$$Y_r \approx \frac{\sum_i X_r(t_i) (t_{i+1} - t_i)}{T - t_w^{\text{final}}}$$

- Altre metriche:

$$Y_r = \frac{1}{n_r} \sum_{i=1}^{n_r} X_{r,i}$$

Dopodiché, la stima puntuale è stata calcolata come **media tra repliche**:

$$\bar{Y} = \frac{1}{R} \sum_{r=1}^R Y_r$$



Calcolo degli intervalli di confidenza

Calcolo della **varianza campionaria** e della **deviazione standard** tra repliche:

$$S^2 = \frac{1}{R-1} \sum_{r=1}^R (Y_r - \bar{Y})^2 \quad S = \sqrt{S^2}$$

Un **intervallo di confidenza** è un intervallo casuale che, con probabilità $1-\alpha$, contiene il vero valore del parametro di interesse.

Poiché la varianza reale della metrica è ignota e disponiamo unicamente della varianza campionaria, per stimare tali intervalli si utilizza la **t di Student**, che tiene conto di questa incertezza.

L'intervallo di confidenza al livello $1 - \alpha$ è pertanto dato da:

$$\bar{Y} \pm t_{1-\alpha/2, R-1} \frac{S}{\sqrt{R}}$$

dove $t_{1-\alpha/2, R-1}$ rappresenta il quantile della distribuzione t di Student calcolato sulla base del numero di repliche disponibili.

Risultati (1)

Tabella 4: Stime puntuali, deviazione standard e intervalli di confidenza (95%) per ciascuna misura di prestazione, per $m = 1.0$

Metrica	Stima puntuale	Deviazione std.	Intervallo di confidenza
Numero utenti (classe 1)	12.4493	0.1241	[12.3912 , 12.5074]
Numero utenti (classe 2)	24.9162	1.1932	[24.3578 , 25.4746]
Durata vacation	2.9641	0.0268	[2.9515 , 2.9766]
Fattore di utilizzo	0.8549	0.0070	[0.8516 , 0.8581]
T. permanenza (classe 1)	4.9645	0.0412	[4.9453 , 4.9838]
T. permanenza (classe 2)	12.4309	0.5592	[12.1692 , 12.6926]

Tabella 5: Stime puntuali, deviazione standard e intervalli di confidenza (95%) per ciascuna misura di prestazione, per $m = 1.4$

Metrica	Stima puntuale	Deviazione std.	Intervallo di confidenza
Numero utenti (classe 1)	11.8868	0.0988	[11.8406 , 11.9331]
Numero utenti (classe 2)	15.6964	0.3363	[15.5390 , 15.8538]
Durata vacation	2.9541	0.0185	[2.9455 , 2.9628]
Fattore di utilizzo	0.6122	0.0073	[0.6088 , 0.6156]
T. permanenza (classe 1)	6.6510	0.0649	[6.6207 , 6.6814]
T. permanenza (classe 2)	10.9988	0.2084	[10.9013 , 11.0963]

Risultati (2)

Tabella 6: Stime puntuali, deviazione standard e intervalli di confidenza (95%) per ciascuna misura di prestazione, per $m = 2.0$

Metrica	Stima puntuale	Deviazione std.	Intervallo di confidenza
Numero utenti (classe 1)	11.4671	0.0790	[11.4301 , 11.5041]
Numero utenti (classe 2)	13.3050	0.2403	[13.1925 , 13.4174]
Durata vacation	2.9489	0.0255	[2.9370 , 2.9609]
Fattore di utilizzo	0.4210	0.0054	[0.4185 , 0.4236]
T. permanenza (classe 1)	9.1248	0.0881	[9.0836 , 9.1661]
T. permanenza (classe 2)	13.3224	0.1899	[13.2335 , 13.4112]

Tabella 7: Stime puntuali, deviazione standard e intervalli di confidenza (95%) per ciascuna misura di prestazione, per $m = 2.5$

Metrica	Stima puntuale	Deviazione std.	Intervallo di confidenza
Numero utenti (classe 1)	11.2195	0.0755	[11.1842 , 11.2548]
Numero utenti (classe 2)	12.5134	0.2768	[12.3839 , 12.6430]
Durata vacation	2.9544	0.0267	[2.9419 , 2.9669]
Fattore di utilizzo	0.3318	0.0048	[0.3295 , 0.3341]
T. permanenza (classe 1)	11.2403	0.1750	[11.1584 , 11.3222]
T. permanenza (classe 2)	15.5588	0.2879	[15.4241 , 15.6936]

Osservazioni

All'aumentare di m :

- il numero medio di utenti nel sistema diminuisce per entrambe le classi;
- il fattore di utilizzo diminuisce significativamente;
- è probabile che il server vada in vacation più frequentemente → tempo medio di permanenza degli utenti nel sistema più lungo.

Controllo della coerenza dei risultati

Per ogni m , il fattore di utilizzo è inferiore a 1; poiché nel sistema è presente un solo server, questo è sufficiente per poter considerare **stabile** il sistema.

Inoltre, verifichiamo che il sistema **rispetta la legge di Little** $L_i = \lambda_i W_i$

Consideriamo il caso $m = 1.0$.

Calcoliamo il tasso medio di arrivo degli utenti:

$$\begin{array}{ll} E[B_1] = \frac{1+4}{2} = 2.5 & \longrightarrow \lambda_1 = 1 \cdot 2.5 = 2.5 \\ E[B_2] = \frac{1+3}{2} = 2 & \lambda_2 = 1 \cdot 2 = 2 \end{array}$$

Dati i tempi medi di permanenza W_1 e W_2 stimati dalla simulazione, i valori previsti del numero medio di utenti nel sistema sono:

$$L_1^{\text{pred}} = \lambda_1 \cdot W_1 = 2.5 \cdot 4.9645 \approx 12.411,$$

$$L_2^{\text{pred}} = \lambda_2 \cdot W_2 = 2 \cdot 12.4309 \approx 24.862.$$

che risultano molto vicini alle stime puntuali della simulazione:

$$L_1 = 12.4493, \quad L_2 = 24.9162$$

References

- [1] P. Karan and S. Pradhan. Analysis of a queue-length-dependent vacation queue with bulk service, n-policy, set-up time and cost optimization. Perform. Evaluation, 167:102459, 2025.
- [2] OpenSim Ltd. OMNeT++ 6.0 Simulation Manual. <https://doc.omnetpp.org/omnetpp/>, 2023.
- [3] Steven S. Lavenberg. Chapter 6 - the statistical analysis of simulation results. In Computer Performance Modeling Handbook, pages 281–312. Academic Press, 1983.
- [4] Christos G. Cassandras and Stephane Lafortune. Introduction to Discrete Event Systems. Springer, 2 edition, 2008.